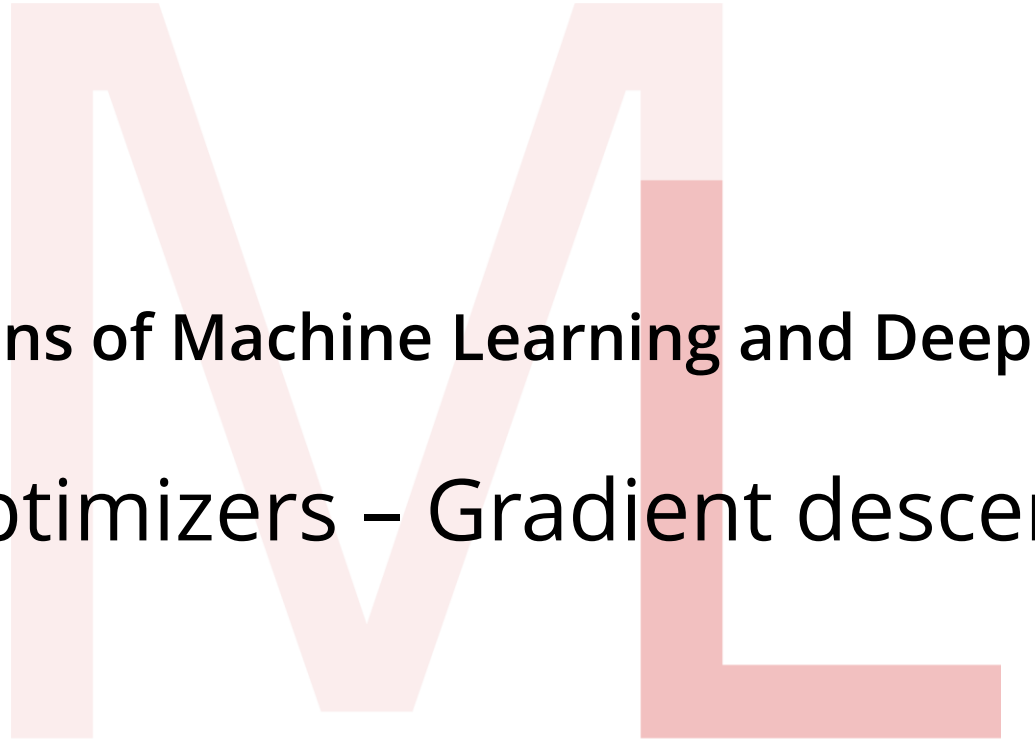
Foundations of Machine Learning and Deep Learning

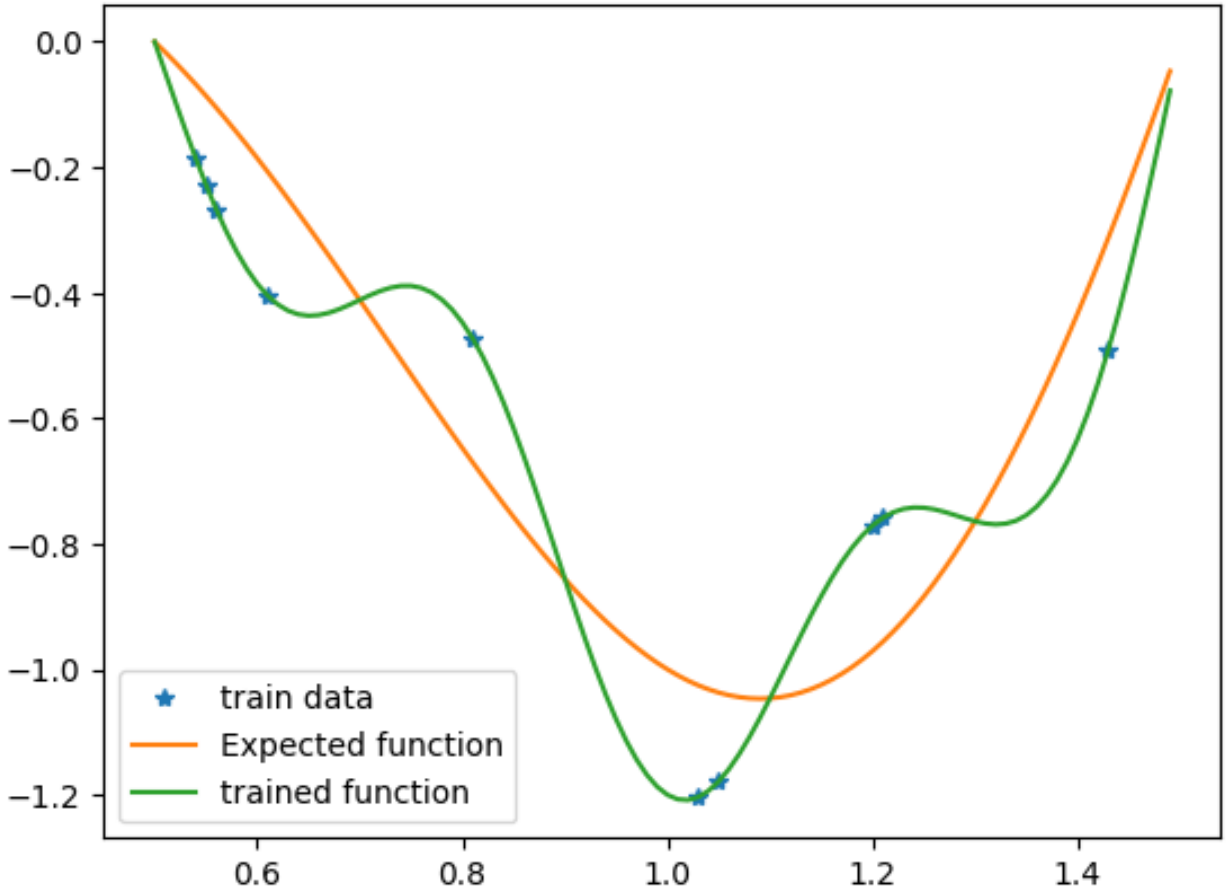
Optimizers – Gradient descent



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Optimization vs Deep Learning

- Optimization
  - to minimize an objective function
  - To reduce the training error
- Deep Learning
  - to find a suitable model
  - To reduce the generalization error (optimization + attention to overfitting)
- Minimum can be located at different point



Optimization

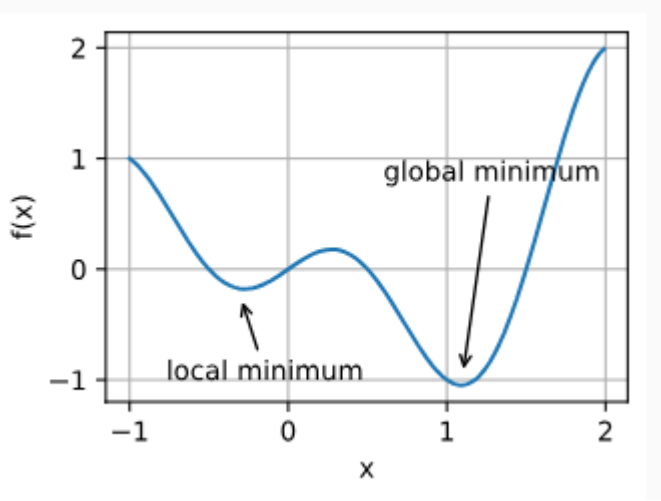
- In deep learning, most objective functions are complicated
  - Do not have analytical solutions
  - To use numerical optimization algorithms

- Most vexing challenges in optimization:
  - Local minima
  - Saddle points
  - Vanishing gradients

Local minima

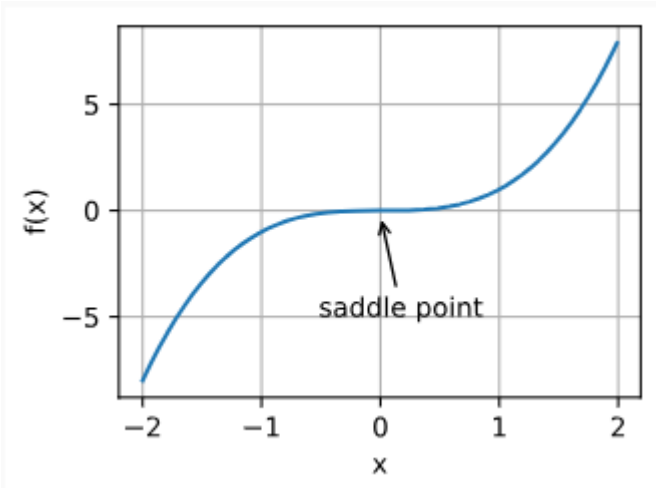
- Local minimum: the value of $f(x)$ at x is smaller tan the values of $f(x)$ at any other points in the vicinity of x
- Global minimum: The value of $f(x)$ at x is the minimum of the objective function over the entire domain
- The objective function of Deep Learning models usually has many local optima

- Example: $f(x) = x \cdot \cos(\pi x)$ for $-1.0 \leq x \leq 2.0$



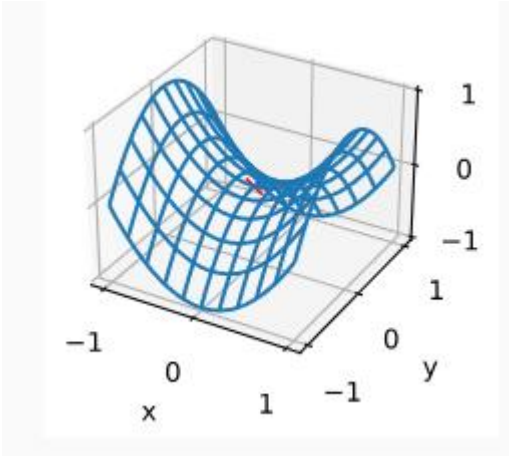
Saddle points

- Saddle point: location where all gradients of a function vanish but which is neither a global nor a local minimum
- Example: $f(x) = x^3$



Saddle points

- Saddle points in higher dimensions are even more insidious
  - At critical points (where the gradient is 0)
    - If all the eigenvalues of the Hessian matrix are positive → local minimum
    - If all the eigenvalues of the Hessian matrix are negative → local maximum
    - If there are positive and negative eigenvalues in the Hessian matrix → saddle point
- Example: $f(x, y) = x^2 - y^2$

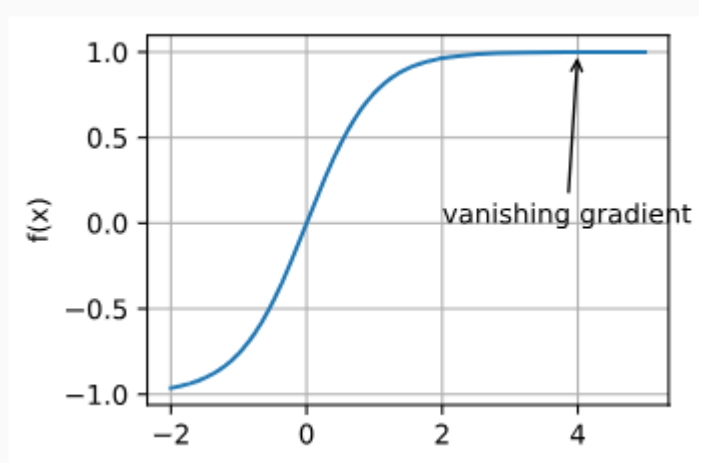


Saddle points

- Example: $f(x, y) = x^2 - y^2$
  - Gradient: $\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right) = (2x, -2y)$
  - Critical points: $\nabla f = 0 \rightarrow (0,0)$
  - Hessian matrix: $\begin{pmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$
  - Eigenvalues: $\det(H - \lambda I) = 0 \rightarrow \lambda = 2;\ \lambda = -2$
  - Eigenvalues are positive and negative → saddle point

Vanishing gradients

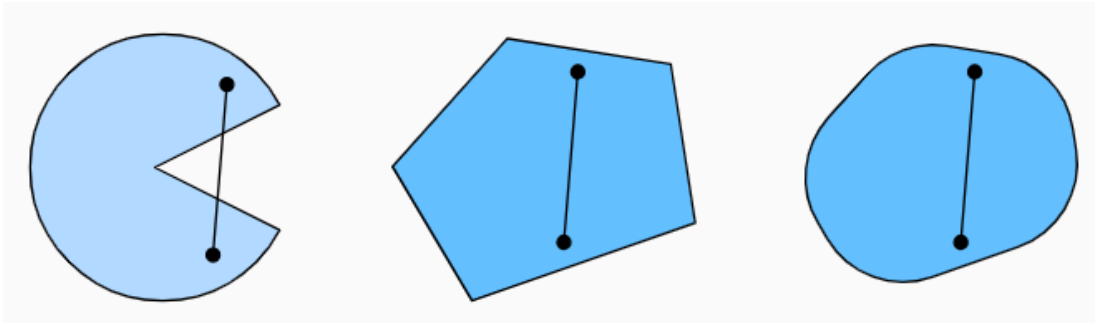
- When gradients tend to 0, they can be stuck for a long time before making progress
- Example: $f(x) = \tanh(x)$
  - $f'(x) = 1 - \tanh^2(x)$
  - In we start at $x = 4$, the gradient is so small it is almost not moving
  - $f'(4) = 0{,}0013$



Convexity

- Convexity plays a vital role in the designing of optimization algorithms, because it is much easier to analyze and test
  - If it does not work in the convex setting, it is not good in general
- Optimization problems in Deep Learning are generally non convex, but they exhibit some properties of convex ones near local minima

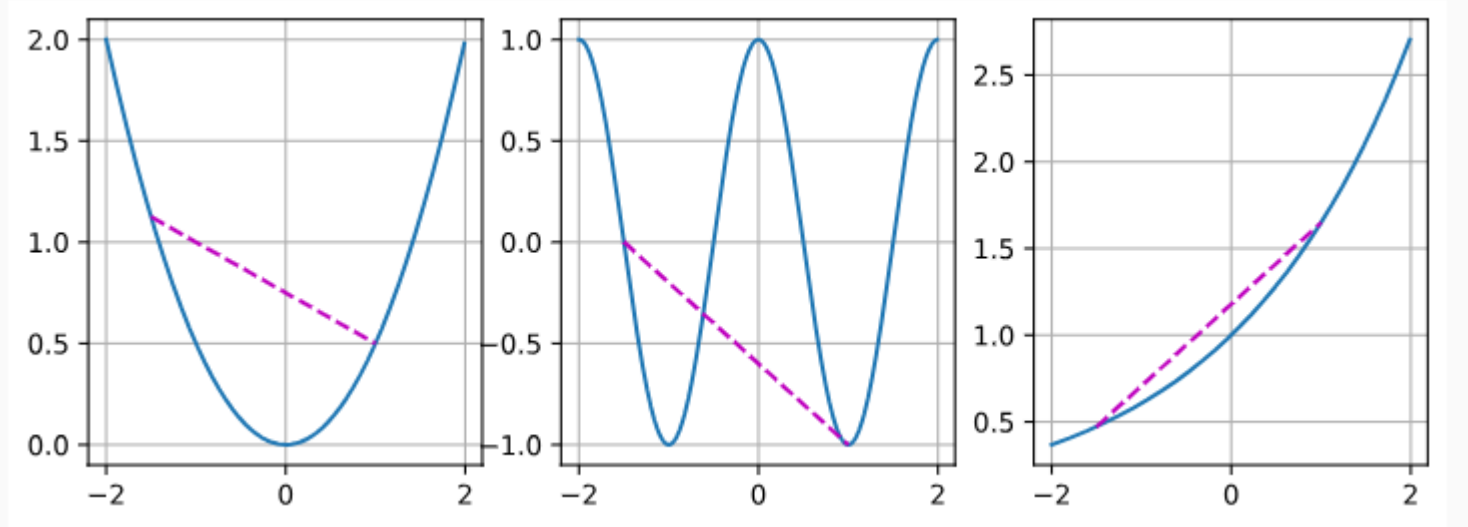
- A set χ in a vector space is convex if for any $a, b \in \chi$ the line segment connecting a and b is also in χ. For all $\lambda \in [0,1]$, $\lambda a + (1-\lambda)b \in \chi$, whenever $a, b \in \chi$



Convexity

- Given a convex set χ, a function $f: \chi \to \mathbb{R}$ is convex if for all $x, x' \in \chi$ and for all $\lambda \in [0,1]$ we have

$$\lambda f(x) + (1-\lambda) f(x') \geq f(\lambda x + (1-\lambda) x')$$



Convexity

- Local minima of convex functions are also the global minima
- Demonstration: by contradiction
  - $x^* \in \chi$ is a local minimum → $\exists p,\ 0 < |x - x^*| \leq p,\ f(x^*) < f(x)$, for $x \in \chi$
  - x^* is not the global minimum → $\exists x',\ f(x') < f(x^*)$
  - z is a point in the segment $x^* - x'$ → $z = \lambda x' + (1 - \lambda)x^*,\ z \in \chi\ \ \forall \lambda \in [0,1]$
  - f is convex, so $f(z) \leq \lambda f(x') + (1 - \lambda) f(x^*) = f(x^*) + \lambda(f(x') - f(x^*))$
  - $f(x') - f(x^*) < 0 \quad \rightarrow \quad f(z) < f(x^*)$
  - Specifically for λ so that $|z - x^*| \leq p$, $f(x^*) < f(z)$ and $f(z) < f(x^*)$ → CONTRADICTION

Convexity

- A twice-differentiable one-dimensional function $f: \mathbb{R} \to \mathbb{R}$ is convex if and only if its second derivative $f'' \geq 0$
- For any twice-differentiable multidimensional function $f: \mathbb{R}^n \to \mathbb{R}$ is convex if and only if its Hessian $\nabla^2 f \succcurlyeq 0$ (is positive semidefinite)
- Demonstration (not necessary for this course)
- Example: $f(x, y) = x^2 + y^2$
  - Hessian matrix: $\begin{pmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$
  - Possitive semidefinite? → $(x_1 \quad x_2)\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 2x_1^2 + 2x_2^2 > 0$ → YES

Taylor Series

- The Taylor series provides a method to approximate the function $f(x)$ if we are given values for the first n derivatives at a point x_0. The idea will be to find a degree n polinomial that matches all the given derivatives at x_0

$$P_n(x) = \sum_{i=0}^{n} \frac{f^{(i)}(x_0)}{i!}(x - x_0)^i$$

- **Example**
  - Approximate the function $f(x) = \ln(x)$
  - Derivatives
    - $f'(x) = \frac{1}{x}$
    - $f''(x) = -\frac{1}{x^2}$
    - $f'''(x) = \frac{2}{x^3}$

Taylor Series

$$P_n(x) = \sum_{i=0}^{n} \frac{f^{(i)}(x_0)}{i!}(x - x_0)^i$$

- **Example**
  - Approximate the function $f(x) = \ln(x)$
  - Polinomial approximations (at point $x_0 = 2$)
    - $P_0(x) = \ln(2)$
    - $P_1(x) = \ln(2) + \frac{1}{2}(x - 2)$
    - $P_2(x) = \ln(2) + \frac{1}{2}(x - 2) - \frac{1}{8}(x - 2)^2$
    - $P_3(x) = \ln(2) + \frac{1}{2}(x - 2) - \frac{1}{8}(x - 2)^2 + \frac{1}{24}(x - 2)^3$

Taylor Series

- **Example**
  - Approximate the function $f(x) = \ln(x) \rightarrow f'(x) = \frac{1}{x}$
  - Polinomial approximations (at point $x_0 = 2$)
    - $P_1(x) = \ln(2) + \frac{1}{2}(x-2)$
      - $\frac{\partial P_1}{\partial x} = \frac{1}{2} \quad \rightarrow \quad \frac{\partial P_1}{\partial x}(x_0) = \frac{1}{2}$
    - $P_2(x) = \ln(2) + \frac{1}{2}(x-2) - \frac{1}{8}(x-2)^2$
      - $\frac{\partial P_2}{\partial x} = \frac{4-x}{4} \quad \rightarrow \quad \frac{\partial P_2}{\partial x}(x_0) = \frac{2}{4}$
    - $P_3(x) = \ln(2) + \frac{1}{2}(x-2) - \frac{1}{8}(x-2)^2 + \frac{1}{24}(x-2)^3$
      - $\frac{\partial P_3}{\partial x} = \frac{x^2-6x+12}{8} \quad \rightarrow \quad \frac{\partial P_3}{\partial x}(x_0) = \frac{4}{8}$

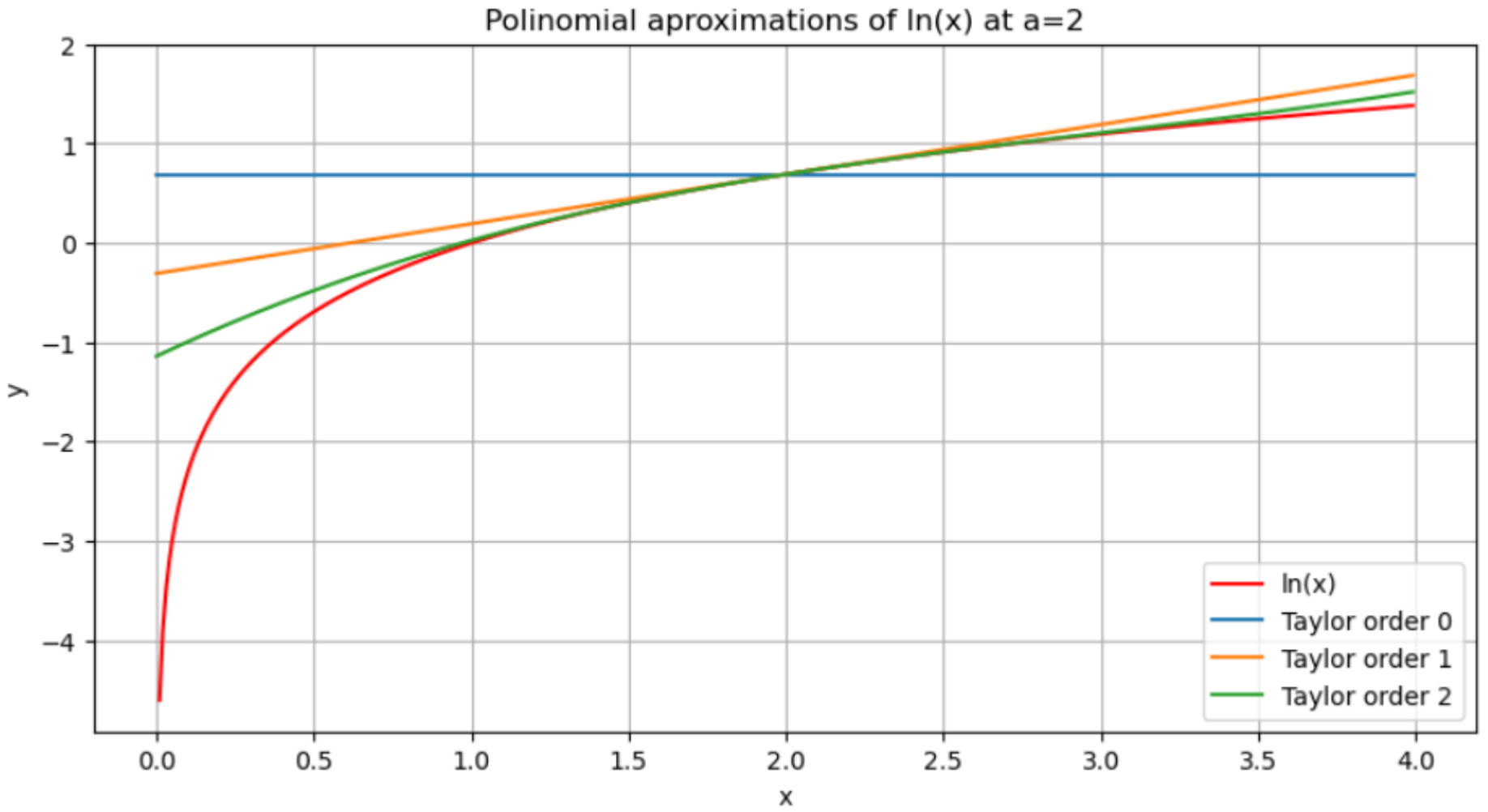
Taylor Series

- **Example**
  - Approximate the function $f(x) = \ln(x)$ → $f''(x) = \frac{-1}{x^2}$
  - Polinomial approximations (at point $x_0 = 2$)
    - $P_2(x) = \ln(2) + \frac{1}{2}(x-2) - \frac{1}{8}(x-2)^2$
      - $\frac{\partial P_2}{\partial x} = \frac{-1}{4}$ → $\frac{\partial P_2}{\partial x}(x_0) = \frac{-1}{4}$
    - $P_3(x) = \ln(2) + \frac{1}{2}(x-2) - \frac{1}{8}(x-2)^2 + \frac{1}{24}(x-2)^3$
      - $\frac{\partial P_3}{\partial x} = \frac{x-3}{4}$ → $\frac{\partial P_3}{\partial x}(x_0) = \frac{-1}{4}$

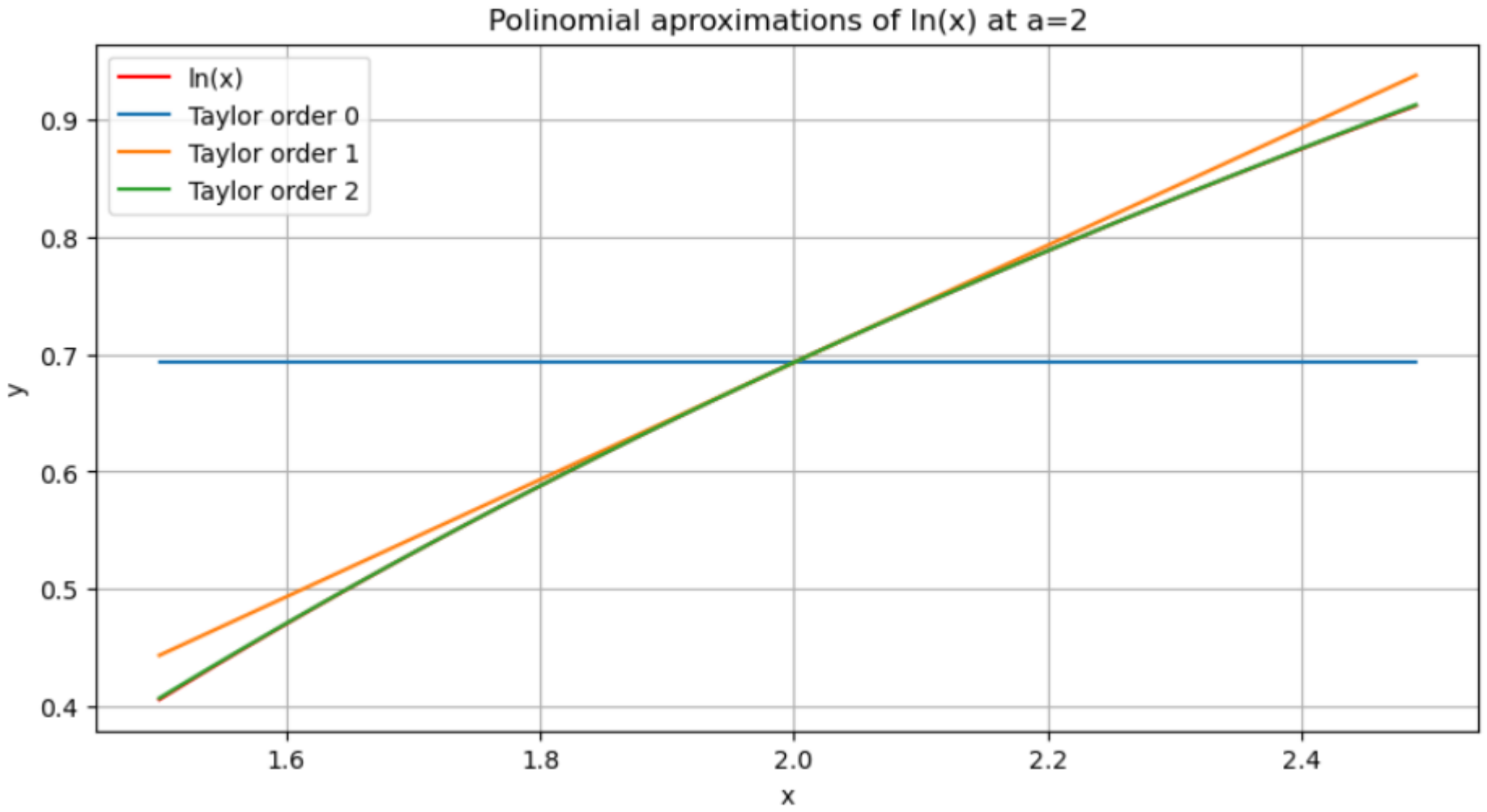
Taylor Series

- **Example**
  - Approximate the function $f(x) = \ln(x)$ → $f'''(x) = \frac{2}{x^3}$
  - Polinomial approximations (at point $x_0 = 2$)
    - $P_3(x) = \ln(2) + \frac{1}{2}(x-2) - \frac{1}{8}(x-2)^2 + \frac{1}{24}(x-2)^3$
      - $\frac{\partial P_3}{\partial x} = \frac{1}{4}$ → $\frac{\partial P_3}{\partial x}(x_0) = \frac{1}{4}$

Taylor Series



Taylor Series

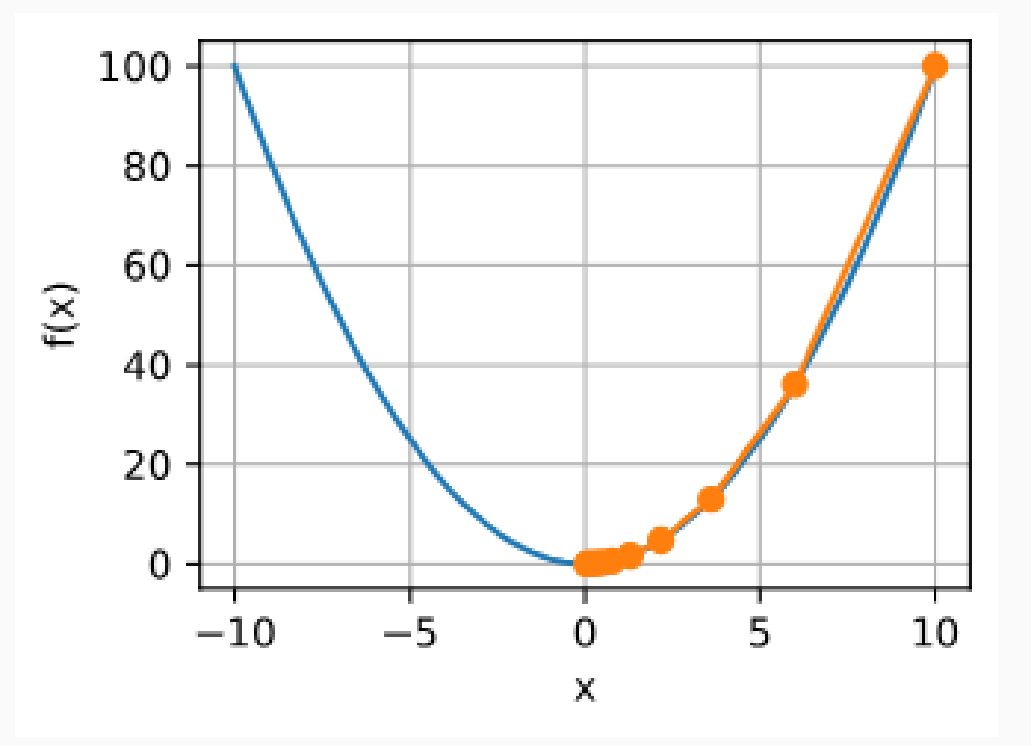


One-dimensional gradient descent

- Consider some continuously differentiable real-valued function $f: \mathbb{R} \to \mathbb{R}$
- Using Taylor (first order): $P_1(x) = f(x_0) + f'(x_0)(x - x_0)$
- Rewriting: $f(x + \epsilon) = f(x) + \epsilon f'(x)$
- Intuitively, to decrease f, we move in the direction of the negative gradient. Is it true?
  - $\epsilon = -\eta f'(x)$
  - $f\big(x - \eta f'(x)\big) = f(x) - \eta f'(x) f'(x) = f(x) - \eta\big(f'(x)\big)^2$
  - If the derivative $f'(x) \neq 0$ then $\eta\big(f'(x)\big)^2 > 0$, so $f\big(x - \eta f'(x)\big) < f(x)$
- We choose an initial value x and a constant $\eta > 0$ and then use them to continuously iterate x until the stop condition is reached (gradient small enough or number of iterations)

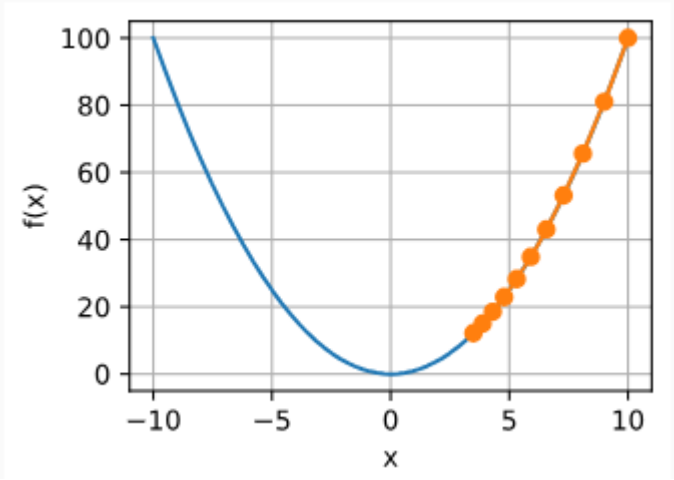
One-dimensional gradient descent

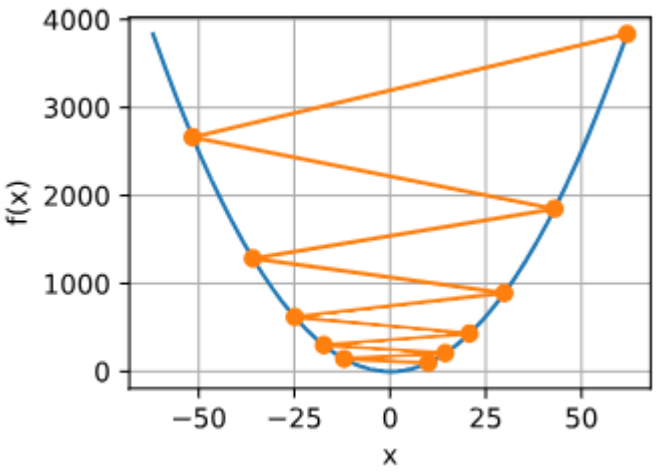
- Example:
  - $f(x) = x^2$
  - $f'(x) = 2x$
  - Initial point: $x = 10$
  - $\eta = 0.2$



One-dimensional gradient descent

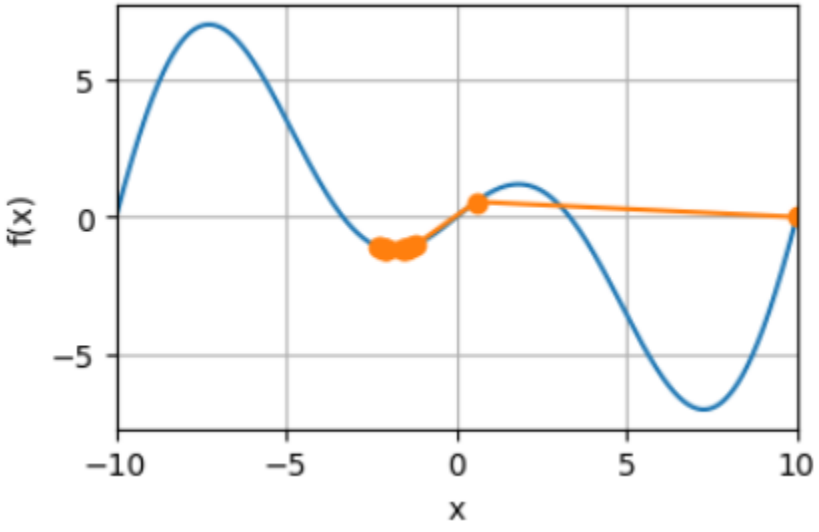
- Learning rate
  - Too small = Too slow convergence
  - Too big = Not guarantee to lower the value of $f(x)$
    - Taylor expansion has a square term considered irrelevant because it is very small (if α is big, it is not irrelevant)
- Example:
  - $f(x) = x^2$
  - Initial point: $x = 10$
  - Iterations = 10
  - $\eta = 0.2$; $\eta = 1.1$

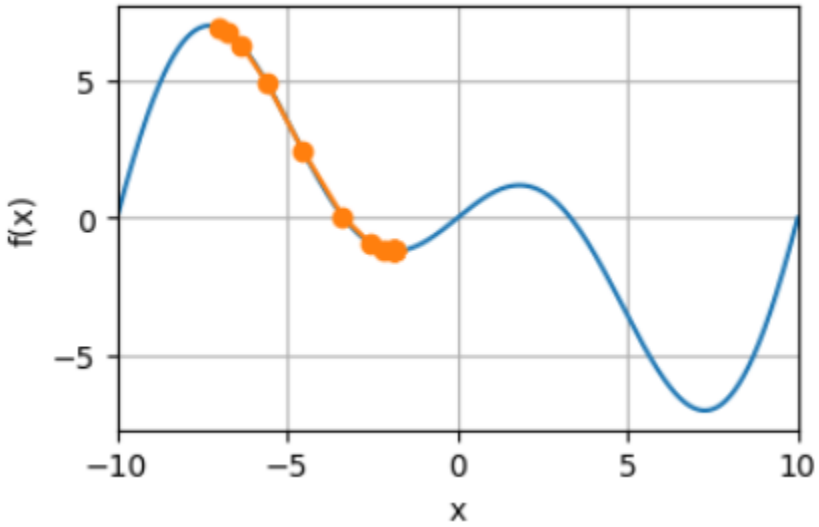


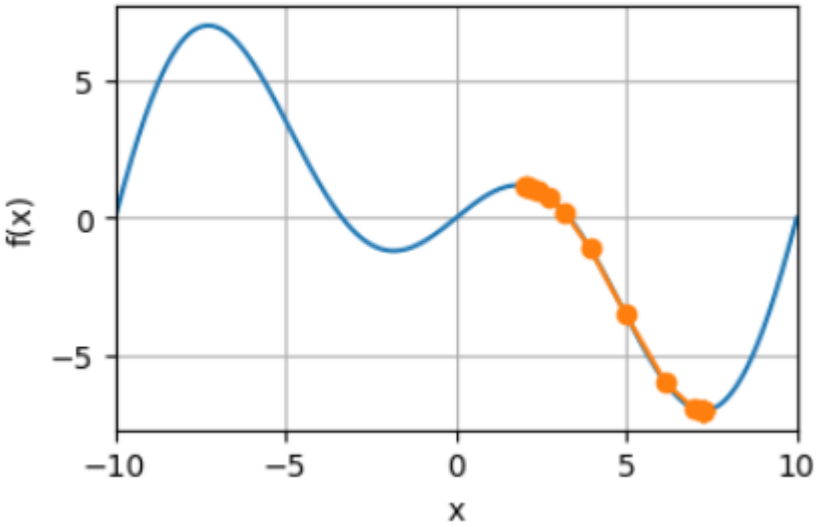


One-dimensional gradient descent

- What happens if the function is not convex
- Example
  - $f(x) = x \cdot \cos(0.15\pi x)$
  - Initial point: $x = 10; \eta = 2$
  - Initial point: $x = -7; \eta = 0{,}5$
  - Initial point: $x = 2; \eta = 0{,}5$







Multivariate gradient descent

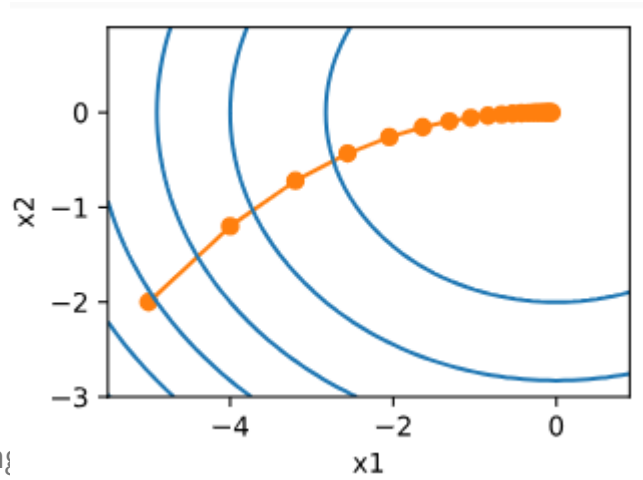
- The objective function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ maps vectors into scalars
- Gradient made of d partial derivatives: $\nabla f(\boldsymbol{x}) = \left[\frac{\partial f(\boldsymbol{x})}{\partial x_1}, \frac{\partial f(\boldsymbol{x})}{\partial x_2}, \ldots, \frac{\partial f(\boldsymbol{x})}{\partial x_d}\right]^T$
- Taylor approximation

$$f(\boldsymbol{x} + \boldsymbol{\epsilon}) = f(\boldsymbol{x}) + \boldsymbol{\epsilon}^T \nabla f(\boldsymbol{x})$$

- Up to second-order terms, in $\boldsymbol{\epsilon}$ the direction of the steepest descent is given by the negative gradient

$$\boldsymbol{x} \leftarrow \boldsymbol{x} - \eta \nabla f(\boldsymbol{x})$$

- Example: $f(\boldsymbol{x}) = x_1^2 + 2x_2^2$
  - Initial point: (-5, -2)
  - $\eta = 0.1$



Newton's Method

- Getting the learning rate η is tricky
- Second-order methods look at the curvature of the function (at the gradient too)
  - Not applicable to deep learning directly due to its computational cost

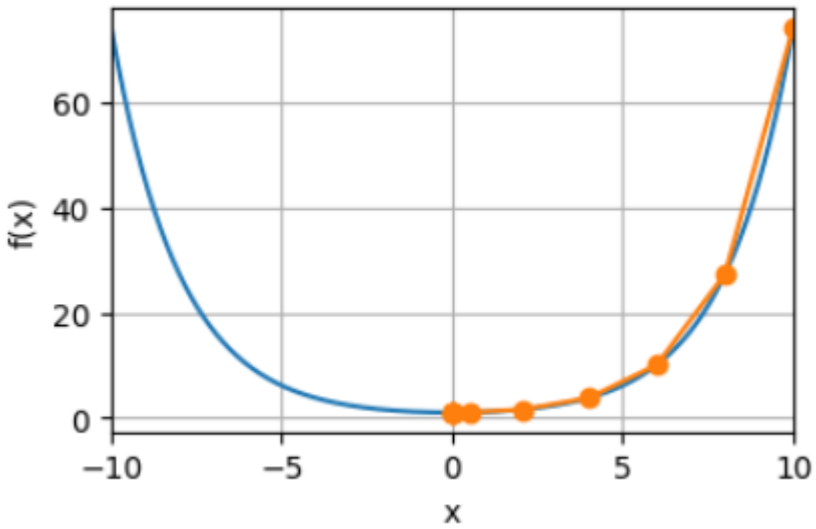
- Taylor second-order polynomial

$$f(\boldsymbol{x}+\boldsymbol{\epsilon}) = f(\boldsymbol{x}) + \boldsymbol{\epsilon}^T \nabla f(\boldsymbol{x}) + \frac{1}{2}\boldsymbol{\epsilon}^T \nabla^2 f(\boldsymbol{x})\boldsymbol{\epsilon}$$

- Find the ϵ that minimizes this function $[\nabla^2 f(\boldsymbol{x}) = \mathrm{H}$ (Hessian)]
  - Differentiate with respect to $\boldsymbol{\epsilon}$: $\frac{\partial f(\boldsymbol{x}+\boldsymbol{\epsilon})}{\partial \epsilon} = \nabla f(\boldsymbol{x}) + \mathrm{H}\boldsymbol{\epsilon}$
  - Set equal to 0: $\boldsymbol{\epsilon} = -H^{-1}\nabla f(\boldsymbol{x})$
    - Invert the Hessian is part of the optimization problem

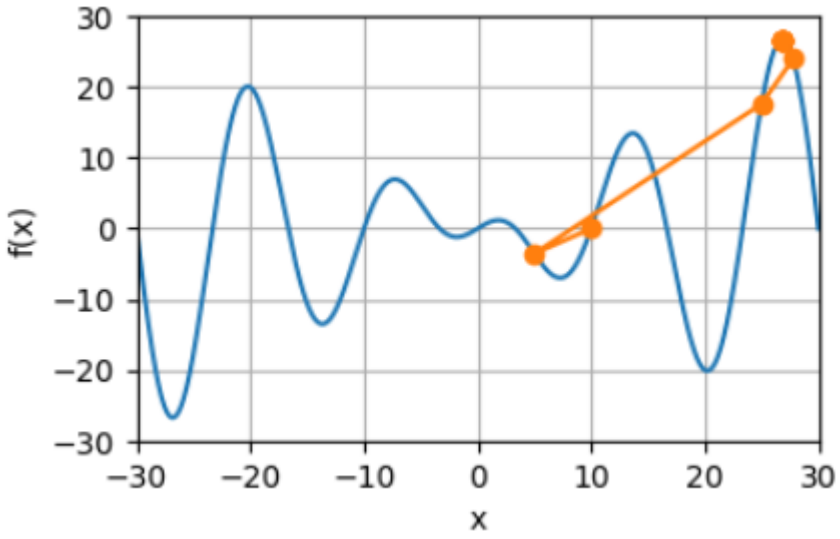
Newton's Method

- When $f(x)$ is an univariate function
  - Differentiate with respect to ϵ: $\frac{\partial f(x+\epsilon)}{\partial \epsilon} = f'(x) + f''(x)\epsilon$
  - Set equal to 0: $\epsilon = -\frac{f'(x)}{f''(x)}$
- Example: $f(x) = \cosh(0.5x)$
  - Initial value: $x = 10$
  - Similar to gradient descent, call η the step
    - Iter1: $\eta = 0.0539$
    - Iter2: $\eta = 0.1465$
    - Iter3: $\eta = 0.3970$
    - Iter4: $\eta = 1.0574$
    - Iter5: $\eta = 2.5111$
    - ...



Newton's Method

- What happens if the second derivative is negative?
  - We may walk into the direction of increasing the value of f
- Example:
  - $f(x) = x \cdot \cos(0.15\pi x)$
  - $f'(x) = \cos(0.15\pi x) - 0.15\pi x \cdot \sin(0.15\pi x)$
  - $f''(x) = -2 \cdot 0.15\pi \cdot \sin(0.15\pi x) - (0.15\pi)^2 x \cdot \cos(0.15\pi x)$
  - Initial point: $x = 10$
  - Update movements:
    - Iter1: $-\frac{f'(x)}{f''(x)} = -5$
    - Iter2: $-\frac{f'(x)}{f''(x)} = 19.9949$
    - Iter3: $-\frac{f'(x)}{f''(x)} = 2.7882$



Newton's method

- Using the Hessian deals with variables in different scales
  - A fixed step is not suitable for different scales (mm, km) → GD is inneficient
  - Hessian measures the curvature in each direction, so adjust the Learning rate in each coordinate
  - Calculate and store the Hessian is very expensive
- With preconditioning, it only computes the diagonal entries of the Hessian
  - It is not as good as the full Newton's method
  - It is able to select different Learning rates for each variable

$$\boldsymbol{x} \leftarrow \boldsymbol{x} - \epsilon\, diag(\boldsymbol{H})^{-1} \nabla f(\boldsymbol{x})$$

Gradient descent with line search

- Move in the direction denoted by the gradient
- The amount of movement is determined by a binary search
  - Determine the maximum value of α ($range = 0..\max_\alpha$)
  - Repeat until the interval is sufficient small
    - Evaluate in the middle of the range of α
    - Determine in which half of the range is the mínimum
    - Update the rango
- Not applicable in deep learning:
  - Each step of the line search requires to evaluate the objective function of the entire dataset

Stochastic Gradient Descent

- Usual loss function in deep learning: average of loss functions of each example

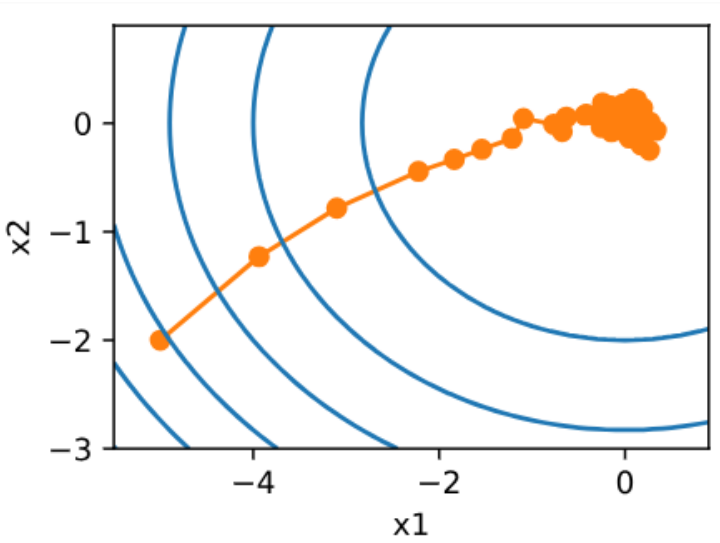
$$f(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{x}) \quad \rightarrow \quad \nabla f(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\boldsymbol{x})$$

- Cost: $O(n)$ → need to reduce when training set large
- Stochastic Gradient Descent
  - Calculate the gradient with respect to one random example at a time
  - Update: $\boldsymbol{x} \leftarrow \boldsymbol{x} - \eta \nabla f_i(\boldsymbol{x})$
  - $O(1)$
  - On average, it is a good estimation of the gradient

$$\mathbb{E}_i \nabla f_i(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{x}) = \nabla f(\boldsymbol{x})$$

Stochastic Gradient Descent

- Example
  - $f(x_1, x_2) = x_1^2 + 2x_2^2$
  - Initial point: (-5, -2)



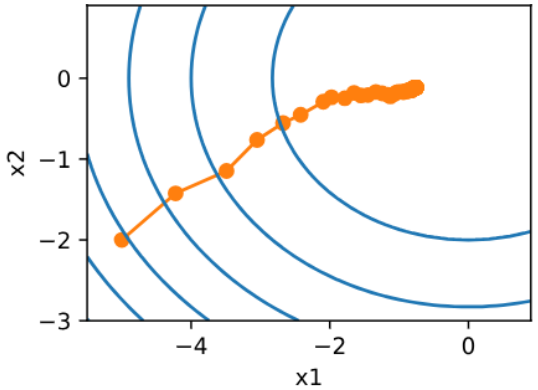
- Trajectories are more noisy
- Near the minimum it is difficult to reach it
  - Reduce the learning rate?

Stochastic Gradient Descent

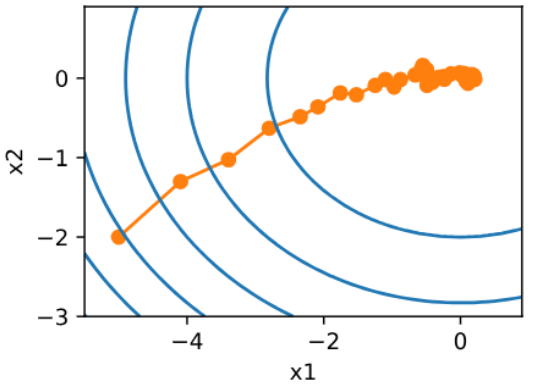
- Using a dynamic learning rate. How rapidly it should decay?
  - Too quick → stop optimizing prematurely
  - Too slow → waste too much time on optimization

- Strategies:
  - Piecewise constant: $\eta(t) = \eta_i$ if $t_i \leq t \leq t_{i+1}$
  - Exponential decay: $\eta(t) = \eta_0 \cdot e^{-\lambda t}$
  - Polynomial decay: $\eta(t) = \eta_0 \cdot (\beta t + 1)^{-\alpha}$

Exponential
$\lambda = 0.1$



Polynomial
$\beta = 0.1$
$\alpha = 0.5$



Minibatch Stochastic Gradient Descent

- Gradient descent – uses full training dataset to compute gradients
- Stochastic gradient descent – uses one training example to compute gradients
- Minibatch stochastic gradient descent – in between
  - As efficient as full gradient descent
    - vector-matrices operations
    - use of paralelization: GPUs and nodes

- The expectation os stochastic gradient descent is the same as the one of minibatch stachastic gradient descent
- The variance of minibatch stochastic gradient descent is reduced by a factor $b^{-\frac{1}{2}}$ (b is the batch size)